CS780 Capstone Project: OBELIX: the Warehouse Robot

Mrityunjay Kumar
230668
mrityunjay23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This project studies reinforcement learning for the OBELIX warehouse robot, whose task is to find a grey box, attach to it, and push it to the arena boundary under increasing partial observability. I started with tabular and value-based baselines, then moved to stacked and recurrent agents, and finally submitted a vectorized discrete Soft Actor-Critic (SAC) agent with twin critics and Random Network Distillation (RND) exploration. I also explored curriculum learning and wall-obstacle training. The final system learned partial box-finding and attachment behavior, but obstacle-aware navigation and stable pushing remained the main bottlenecks.

1 Competition Result

Codabench Username: m_230668
Final leaderboard rank (Test Phase): 71
Leaderboard rank – Level 1 (static): 19
Leaderboard rank – Level 2 (blinking): 57
Leaderboard rank – Level 3 (moving + blinking): 83
GitHub repository: github.com/mrityunjaykumar23/Cs780_OBELIX
YouTube demonstration video (final robot behaviour): youtube.com/shorts/aTJtBOnnDDA

Table 1: Number of archived Codabench score snapshots available for each phase.

Phase	Number of submissions
Phase 1 result archive	7
Phase 2 result archive	6
Phase 3 result archive	4
Test phase result archive	3

2 Introduction and Problem Description

OBELIX is a warehouse-style robot control problem where an agent must navigate and manipulate a target object using only a compact binary sensor state in a partially observable environment. At every step, the agent receives an 18-dimensional observation derived from sonar and infrared-style

signals and must choose one of five discrete actions: `L45`, `L22`, `FW`, `R22`, and `R45`. The goal is to locate the grey box, reach it, attach to it, and then push it to the arena boundary to successfully end the episode.

Although the action space is simple, the task itself is difficult because the robot never observes the full environment directly. It must infer where the box is, how it is moving, and how to approach it using only limited local sensing. This makes the problem especially challenging when the agent loses sight of the box or has to navigate around obstacles with incomplete information.

The environment becomes progressively harder across three difficulty levels. In Level 1 the box is static, so the task mainly tests search and alignment. In Level 2 the box blinks in and out of visibility, which introduces partial observability and makes it harder for the agent to track the target over time. In Level 3 the box both blinks and moves, so the agent must not only search but also react to motion and maintain control under uncertainty. Wall obstacles make the task even harder by forcing the robot to route through narrow openings instead of moving directly toward the target.

The main objective of this project was to design and train an RL agent that could achieve high cumulative reward while remaining robust across these increasingly difficult settings. In practice, the wall-obstacle and difficulty-3 conditions turned out to be the most demanding, since they exposed weaknesses in memory, navigation, and long-horizon box control. For that reason, much of the project focused not just on improving reward, but on developing policies that behaved more reliably in the hardest cases.

3 Approach and Model Evolution

Baseline and early submissions

My initial work did not begin with a learned model. Before training any RL agent, I first tried a heuristic controller to understand how the robot behaves in the environment, how the sensors respond, and what kinds of motion patterns are needed to find and approach the box. This was useful for debugging the simulator and building intuition, but the heuristic policy was too brittle to solve the task reliably, especially once the box became partially visible or obstacles blocked the direct path.

After that exploratory stage, I moved to simple value-based methods to establish trainable baselines and better understand the reward structure. The first archived Phase 1 submission used tabular Q-learning [1] over the 18-bit observation space. This approach was straightforward to implement and served as a useful starting point, but it did not generalize well because the observation space was large, sparse, and only weakly informative about long-term outcomes. The second Phase 1 submission moved to a Neural Fitted Q-style multilayer perceptron (MLP) Q-network [2], which improved function approximation but still struggled with obstacle handling and the higher-difficulty settings. The third Phase 1 submission introduced a DQN/Double-DQN style learner with prioritized replay [3, 4, 5], mainly targeting better sample efficiency and more stable updates.

Intermediate development

In Phase 2 I focused on difficulty 2, where blinking made the problem more clearly partially observable. I explored both actor-critic and value-based upgrades. The archived folder shows an A2C + LSTM model, a PPO/A2C-style recurrent actor-critic variant, and DDQN variants that used frame stacking and task-specific behavior adjustments [6, 7, 8]. These experiments taught an important lesson: once the box is not consistently visible, methods that rely only on the current

18-bit observation tend to lose track of the target. Recurrent or stacked-state agents behaved more sensibly, but they were also harder to stabilize and tune.

Later development and final strategy

For Level 3 I explored recurrent PPO-like training, but eventually the final test-phase submission converged to a vectorized discrete SAC implementation. The final submitted system used a lightweight inference policy backed by a trained SAC checkpoint. The rationale for switching to SAC was that parallel off-policy training could make better use of limited wall-clock time, especially with multiple environments running in parallel.

Table 2: Summary of the main RL algorithms explored during the project.

Phase	Algorithm	Why it was tried	Main outcome
Early exploration	Heuristic controller	To understand sensors, turning dynamics, attachment behaviour, and failure cases before training learned agents	Useful for intuition and debugging, but too brittle for blinking boxes and obstacle cases
Phase 1	Q-learning / NFQ / DQN-style methods	To build value-based baselines over the 18-bit observation space and improve sample efficiency progressively	Established baselines, but struggled to generalize and remained weak in hard settings
Phase 2	A2C, PPO-LSTM, REINFORCE,VPG, DDQN variants	To address partial observability using recurrence, frame stacking, and actor-critic updates	Better temporal behaviour in some cases, but harder to stabilize and tune reliably
Phase 3	Recurrent PPO-like agents	To improve tracking and action consistency under blinking plus motion	Showed some promising behaviour, but training stability and final robustness were limited
Final	Discrete SAC + RND	To exploit parallel off-policy learning, stronger exploration, and stable critic updates	Produced the strongest final submission and best overall archived test-phase performance

Final model

The final model belongs to the discrete actor-critic family. More specifically, it is a vectorized discrete Soft Actor-Critic agent [9] trained with two Q-networks, target critics, and an actor that outputs a categorical distribution over the five available actions. The policy network is a two-hidden-layer MLP with architecture $18 \to 256 \to 256 \to 5$, and the final submission uses greedy action selection at evaluation time. The state representation in the final submitted agent is the raw 18-bit observation vector rather than a hand-crafted feature vector. Exploration during training comes from the maximum-entropy SAC objective together with an additional Random Network Distillation bonus [10]. I also added a curriculum mode in the later training code so that one training run can progress from difficulty 0 to difficulty 2 and then to difficulty 3, while keeping wall obstacles enabled throughout the curriculum.

Unlike some earlier LSTM/PPO experiments, the final SAC branch does not rely on reward shaping. That choice was intentional: I wanted the final evaluation-time behavior to remain aligned

with the original environment reward and avoid a train-test mismatch caused by shaped rewards that are unavailable during testing.

Why the final SAC agent was chosen

- **Better sample usage:** SAC is off-policy, so the replay buffer allowed each collected transition to be reused more effectively than in my on-policy experiments.
- **Faster training throughput:** the final branch worked cleanly with multiple parallel environments, which made overnight training runs more practical on the available CPU-heavy setup.
- **Best final-phase behaviour:** among the archived candidates, the SAC branch gave the strongest combined local and Codabench performance, especially in the later project stages.

4 Experiments and Implementation Details

All main training runs used the OBELIX simulator provided in the project repository. The episode length was typically 1000 steps, with longer development sweeps performed at 800 episodes and final candidate runs performed at 2000 episodes or more. The final SAC branch was trained on CPU using vectorized environments, since the available machine setup was CPU-dominant. In practice, an 800-episode SAC run with four parallel environments took roughly two hours, so full 2000-episode runs were effectively overnight jobs.

The final training pipeline uses four parallel environments, a replay buffer of size 300,000, a discrete SAC update with twin critics and soft target updates, and an RND exploration bonus. I used fixed evaluation settings (`runs=10`, `max_steps=1000`) during local testing to keep comparisons fair and reduce overfitting to one lucky seed.

Table 3: Representative hyperparameters for the final SAC branch.

Component	Value
Algorithm	Discrete SAC + RND
Observation size	18 binary sensor values
Action space	5 discrete actions
Number of parallel environments	4
Episodes	2000 (typical final run)
Max steps per episode	1000
Learning rate	1×10^{-4}
Discount factor γ	0.995
Batch size	128
Replay start threshold	12000 transitions
Updates per step	1
Replay buffer size	300000
Target update coefficient τ	0.005
RND coefficient β	0.02
Curriculum stages	$0 \rightarrow 2 \rightarrow 3$
Wall obstacles in curriculum	True for all stages

I also compared several hyperparameter settings and logged the outcomes during development. The main variations were learning rate, RND strength, replay warmup, update frequency, and batch size. The resulting differences were useful but relatively small, which suggested that better hyperparameters alone were not enough to solve the core difficulty-3 wall-navigation issue.

Reproducibility and final submission

- **Final submitted policy:** lightweight inference wrapper around the trained SAC actor
- **Final submitted model:** SAC checkpoint with greedy action selection at evaluation time
- **Training setup:** vectorized discrete SAC with four parallel environments
- **Evaluation setup:** 10-run local evaluation with 1000-step episodes
- **Compute setup:** CPU-based training with 4 parallel environments
- **Typical runtime:** about 2 hours for 800 episodes and overnight for 2000-episode final runs

5 Results

The Codabench archive is organized by phase, and each submission folder contains a `scores.txt` file. I therefore report the results phase by phase rather than mixing all submissions into a single table. Phase 1 and Phase 2 score files each contain four values: cumulative reward with wall, its standard deviation, cumulative reward without wall, and its standard deviation. Phase 3 adds a weighted cumulative reward, and the test phase reports separate wall/no-wall scores for difficulties 0, 2, and 3.

Table 4: Phase 1 results from `result_codabench/phase_1_result/scoring_result(*)/scores.txt`.

Submission	With Wall	Std (Wall)	No Wall	Std (No Wall)
Submission 1	-176593.3	43899.098	-140725.8	58799.716
Submission 2	-20098.7	8695.291	-32209.6	4457.169
Submission 3	-211573.5	183036.594	-23666.7	16820.589
Submission 4	-17128.1	10768.341	-31180.9	6715.382
Submission 5	-1981.2	7.692	-1990.9	10.445
Submission 6	-1976.0	36.436	-1992.7	8.615
Submission 7	-1991.0	6.971	-1992.8	8.495

These phase-wise tables show the overall trajectory clearly. Phase 1 varied dramatically as I moved from tabular Q-learning toward neural value-based methods. Phase 2 clustered near similar negative scores, suggesting that algorithm swaps alone were not enough to solve partial observability. Phase 3 showed the largest qualitative improvement, with Submission 3 becoming the strongest pre-test candidate.

The test-phase tables confirm the same story. Submission 3 is the strongest stored final candidate with weighted cumulative reward -1832.807, slightly better than Submission 2 and much better than Submission 1. The hardest setting remained difficulty 3 with wall obstacles, where even the best submission still had clearly negative return.

Table 5: Phase 2 results from `result_codabench/phase_2_result/scoring_result(*)/scores.txt`.

Submission	With wall	Std (wall)	No wall	Std (no wall)
Submission 1	-277684.8	180516.548	-1172.7	1618.059
Submission 2	-1985.9	11.406	-1989.5	11.307
Submission 3	-392743.7	6618.110	-24170.2	35133.321
Submission 4	-1981.2	7.692	-1990.9	10.445
Submission 5	-1989.7	9.067	-1989.1	11.734
Submission 6	-1986.7	9.360	-1988.7	12.522

Table 6: Phase 3 results from `result_codabench/phase_3_result/scoring_result(*)/scores.txt`.

Submission	With wall	Std (wall)	No wall	Std (no wall)	Weighted
Submission 1	-358068.3	111122.677	-356793.3	73859.828	-357558.3
Submission 2	-311805.8	156964.823	1842.2	263.120	-186346.6
Submission 3	**-8741.8**	**7834.222**	**-1481.0**	**1197.066**	**-5837.480**
Submission 4	-49989.2	42555.238	-5340.8	19568.516	-32129.840

For local evaluation, the strongest no-wall result in `leaderboard.csv` for difficulty 3 reached a mean score of 1121.7, indicating that the agent could sometimes solve the moving-blinking setting without walls. In contrast, the wall-obstacle setting remained much harder: the best local wall-obstacle evaluation I obtained for the final SAC branch was around mean score -653.2 over 10 runs. This gap is consistent with the visual behavior observed during evaluation videos, where the agent often found or attached to the box but failed to route it reliably through the obstacle geometry.

Overall, the final SAC model performed best among the archived submissions because it combined better sample throughput, stable off-policy learning, and stronger exploration than the earlier baselines. However, the final gains were much more pronounced on easier settings than on the full wall-obstacle task, which suggests that state representation and memory limitations were more important than purely optimizer-level tuning.

6 Error Analysis and Discussion

Failure cases

The most common failure modes were strongly tied to partial observability. In Level 1, failures mostly came from poor alignment or ineffective pushing even after the box was found. In Level 2, blinking caused the agent to lose track of the target and drift into reactive local motion. In Level 3, movement plus blinking made interception difficult: the policy often chased the current visible location instead of predicting where the box would be a few steps later.

With walls enabled, the failure pattern became much more obvious. The agent frequently hovered near the wall opening, rotated repeatedly, and behaved reactively rather than following a stable route around the obstacle. The evaluation video shows exactly this pattern: the robot can often search, approach, and sometimes attach to the box, but it struggles to convert attachment into a reliable push-to-boundary strategy. In other words, the agent has learned meaningful subskills (search and attach) but not a fully obstacle-aware end-to-end solution.

Another important observation is that some reward can be accumulated without truly solving the intended task. For example, approaching the box, maintaining contact briefly, or avoiding

Table 7: Test-phase wall-condition results from `result_codabench/test_phase_result/scoring_result(*)`.

Submission	L0 wall	L2 wall	L3 wall	Weighted
Submission 1	-10161.2	-7551.8	-8881.8	-6136.640
Submission 2	-1991.6	-1988.5	-1537.2	-1834.660
Submission 3	**-1988.5**	**-1987.5**	**-1535.9**	**-1832.807**

Table 8: Test-phase no-wall results from `result_codabench/test_phase_result/scoring_result(*)`.

Submission	L0 no wall	L2 no wall	L3 no wall	Mean no wall
Submission 1	-2325.8	-2325.8	-1481.0	-2044.200
Submission 2	-1996.5	-1989.4	-1498.1	-1828.000
Submission 3	**-1994.2**	**-1989.2**	**-1494.8**	**-1826.067**

certain catastrophic events can improve return even if the box is not pushed to the boundary. This is a useful edge case to highlight because it explains why occasional positive-looking behaviors in the video do not always correspond to consistent task completion.

Lessons learned

- Partial observability was a bigger bottleneck than optimizer choice alone.
- Memory-oriented models helped conceptually, but were harder to train reliably under the available time and compute budget.
- Wall-obstacle navigation remained the clearest gap between partial success and robust task completion.

The main limitations of the final system are therefore: (i) lack of explicit memory in the final submitted SAC policy, (ii) weak route planning around walls, and (iii) unstable long-horizon control after box attachment. If I had more time, the next changes I would prioritize are frame stacking or recurrent SAC, explicit evaluation-based checkpoint selection, and a cleaner decay schedule for the RND bonus.

7 Conclusion

This project showed that the OBELIX task is deceptively difficult once blinking, box motion, and wall obstacles are introduced. Starting from tabular Q-learning and progressively moving toward neural value-based, recurrent actor-critic, and finally discrete SAC agents gave a useful view of how algorithm choice interacts with partial observability. The final SAC submission improved substantially over the earliest baselines and showed meaningful learning of search and attachment behavior, but still lacked robust obstacle-aware planning and stable box pushing in the hardest setting.

The most important lesson from this project is that partially observable robotic tasks often fail for representational reasons rather than only optimization reasons. Better hyperparameters helped, but only up to a point. Memory, temporal abstraction, and robust navigation matter more once the sensor stream stops being fully informative. Future work could explore recurrent off-policy

Table 9: Representative local evaluation results from the development process.

Setting	Mean score	Observation
Difficulty 3, no wall	**1121.7**	Occasional successful completion
Difficulty 3, wall obstacle	**-653.2**	Partial success, unstable pushing
Difficulty 2, no wall	about -989	Generally failed to capitalize on blinking tracking

agents, explicit motion prediction, richer behavior decomposition, or extensions toward more realistic pushing physics and multi-object settings.

LLM Usage Declaration

- **Which Large Language Model (LLM) did you use?**
  OpenAI Codex / GPT-family assistance was used in a limited support role for ideation, code debugging, and report drafting.

- **In which part did you use the LLM, and how did it help you?**

Part	Used LLM?	Purpose of Using LLM
Understanding concepts	Yes	Clarifying algorithm choices such as SAC, PPO, replay-buffer tuning, and curriculum design.
Ideation / brainstorming	Yes	Comparing alternative agent designs and deciding what to try next.
Debugging small code snippets	Yes	Fixing loader mismatches, evaluation rendering, and training-script issues.
Plotting / visualization code	No	Not used for final plotting in this submission.
Grammar / writing style	Yes	Improving clarity, grammar, and report flow.
Report structure / section ideas	Yes	Converting project notes into a clean report structure.
Other (LaTeX drafting)	Yes	Helping convert project findings into LaTeX text.

- **Did you refer to any other sources or websites apart from the LLM and lecture slides?**
  Yes. I referred to the original OBELIX paper, the provided project repository/documentation, and standard RL references for SAC and RND.

- **Apart from any LLM usage, what was your own contribution?**
  I implemented and trained multiple agent families, organized and archived submissions across phases, tuned and compared models on local and Codabench-style evaluations, analyzed failure cases from scores and videos, and selected the final test-phase SAC submission. The core experimentation decisions, code execution, interpretation of results, and final model selection were my own.

References

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] M. Riedmiller, "Neural fitted q iteration – first experiences with a data efficient neural reinforcement learning method," in *European Conference on Machine Learning*, pp. 317–328, 2005.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[5] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *International Conference on Learning Representations*, 2016.

[6] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," *International Conference on Machine Learning*, pp. 1928–1937, 2016.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, pp. 1861–1870, 2018.

[10] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," *arXiv preprint arXiv:1810.12894*, 2019.